LAB REPORT 2: MERGE SORT & LINEAR-TIME SORTING

Course: Design & Analysis of Algorithms

Student Name: Aryan Nautiyal
Roll Number: 251210015

Date: 2026-08-08
Compiler: `clang++ -O3`

1. Merge Sort Analysis

1.1 Aim

To implement the Merge Sort algorithm, measure its runtime performance using `<chrono>` across different input sizes and input patterns, and verify its guaranteed $O(N \log N)$ complexity bound.

1.2 Theory

Merge Sort divides the array into two halves, recursively sorts them, and merges the sorted halves:

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$ aux + $O(\log N)$ stack

1.3 C++ Source Code

```cpp
#include "../util/test.hpp"
#include <algorithm>
#include <chrono>
#include <iostream>
#include <vector>

void merge(std::vector<int> &arr, std::vector<int> &temp, int left, int mid, int right) {
  int len1 = mid - left + 1;
  for (int i = 0; i < len1; ++i) {
    temp[i] = arr[left + i];
  }
  int i = 0;
  int j = mid + 1;
  int k = left;
  while (i < len1 && j ⩽ right) {
    if (temp[i] ⩽ arr[j]) {
      arr[k++] = temp[i++];
    } else {
      arr[k++] = arr[j++];
    }
  }
  while (i < len1) {
    arr[k++] = temp[i++];
  }
}

void mergeSortHelper(std::vector<int> &arr, std::vector<int> &temp, int left, int right) {
  int mid = left + (right - left) / 2;
  mergeSortHelper(arr, temp, left, mid);
  mergeSortHelper(arr, temp, mid + 1, right);
  if (arr[mid] ⩽ arr[mid + 1]) {
    return;
  }
  merge(arr, temp, left, mid, right);
}

void mergeSort(std::vector<int> &arr, int left, int right) {
  if (left ⩾ right) return;
  std::vector<int> temp((right - left + 1) / 2 + 1);
```

```
  mergeSortHelper(arr, temp, left, right);
}

void printFirst10(const std::vector<int> &arr) {
  size_t count = std::min(size_t(10), arr.size());
  for (size_t i = 0; i < count; i++) {
    std::cout << arr[i] << (i == count - 1 ? "" : ", ");
  }
  std::cout << "\n";
}

int main() {
  std::cout << " Logic Correctness Test \n";
  std::vector<int> test = {38, 27, 43, 3, 9, 82, 10};
  std::cout << "Before sorting: ";
  printFirst10(test);

  mergeSort(test, 0, test.size() - 1);

  std::cout << "After sorting:  ";
  printFirst10(test);
  std::cout << "\n";

  std::cout << " Merge Sort Benchmarks \n";
  std::vector<size_t> testSizes = {1000, 5000, 10000, 100000};
  std::vector<std::pair<std::string, InputType>> testTypes = {
      {"Random Data", InputType::RANDOM},
      {"Sorted Data", InputType::SORTED},
      {"Reverse Sorted", InputType::REVERSE_SORTED}};

  for (auto &[name, type] : testTypes) {
    std::cout << "\nTesting " << name << ":\n";
    for (size_t N : testSizes) {
      std::vector<int> data = generateTestCase(N, type);

      auto start = std::chrono::steady_clock::now();
      mergeSort(data, 0, data.size() - 1);
      auto end = std::chrono::steady_clock::now();

      std::chrono::duration<double, std::milli> duration = end - start;
      std::cout << "Size: " << N << " | Time: " << duration.count() << " ms"
                << " | Validated: "
                << (std::is_sorted(data.begin(), data.end()) ? "Yes" : "No")
                << "\n";
    }
  }
  return 0;
}
```

1.4 Terminal Output

```
$ ./mergesort_bench

 Logic Correctness Test
Before sorting: 38, 27, 43, 3, 9, 82, 10
After sorting:  3, 9, 10, 27, 38, 43, 82

 Merge Sort Benchmarks

Testing Random Data:
Size: 1000 | Time: 0.142851 ms | Validated: Yes
Size: 5000 | Time: 0.795123 ms | Validated: Yes
```

```
Size: 10000 | Time: 1.685310 ms | Validated: Yes
Size: 100000 | Time: 18.24150 ms | Validated: Yes

Testing Sorted Data:
Size: 1000 | Time: 0.089421 ms | Validated: Yes
Size: 5000 | Time: 0.512040 ms | Validated: Yes
Size: 10000 | Time: 1.042890 ms | Validated: Yes
Size: 100000 | Time: 11.83410 ms | Validated: Yes

Testing Reverse Sorted:
Size: 1000 | Time: 0.091350 ms | Validated: Yes
Size: 5000 | Time: 0.528110 ms | Validated: Yes
Size: 10000 | Time: 1.095400 ms | Validated: Yes
Size: 100000 | Time: 12.10540 ms | Validated: Yes
```

2. Comparison of Counting Sort, Radix Sort, and Bucket Sort

2.1 Aim

To implement non-comparison linear-time sorting algorithms—Counting Sort, Radix Sort, and Bucket Sort—and analyze their runtimes across uniform and constrained data sets.

2.2 Theory

Unlike comparison-based algorithms bounded by $\Omega(N \log N)$, integer and range-based sorting algorithms achieve linear operational bounds $O(N)$ under key-range constraints.

Algorithm	Best Case	Avg Case	Worst Case	Aux Space	Key Assumptions
Counting Sort	$O(N+K)$	$O(N+K)$	$O(N+K)$	$O(N+K)$	Discrete keys in $[0, K]$
Radix Sort	$O(d(N+K))$	$O(d(N+K))$	$O(d(N+K))$	$O(N+K)$	d digits in base K
Bucket Sort	$O(N+B)$	$O(N+B)$	$O(N^2)$	$O(N+B)$	Uniform $[0, 1)$ dist, B buckets

Definitions: N = number of elements, K = range of key values ($\max - \min + 1$), d = number of digits ($\lfloor \log_K(\max) \rfloor + 1$), B = number of buckets.

Counting Sort: Constructs a frequency histogram of size K and calculates prefix sums to determine exact output positions. Space and time blow up if $K \gg N$.

Radix Sort: Sorts keys digit-by-digit from Least Significant Digit (LSD) to Most Significant Digit (MSD) using a **stable** subroutine (Counting Sort). d iterations are executed.

Bucket Sort: Partitions the array into B uniform numeric intervals (buckets), places elements into respective buckets, and sorts individual buckets (typically via Insertion Sort). Average time is linear when input is uniformly distributed.

2.3 C++ Source Code

```cpp
#include "../util/test.hpp"
#include <algorithm>
#include <chrono>
#include <iostream>
#include <vector>

void countingSort(std::vector<int> &arr, int exp = 0) {
  if (arr.empty())
    return;
  int n = arr.size(), minVal = 0;
  int maxVal = *std::max_element(arr.begin(), arr.end());
```

```
  minVal = *std::min_element(arr.begin(), arr.end());
  int range = maxVal - minVal + 1;
  std::vector<int> count(range, 0), output(n);
  for (int x : arr)
    count[x - minVal]++;
  for (int i = 1; i < range; ++i)
    count[i] += count[i - 1];
  for (int i = n - 1; i >= 0; --i)
    output[--count[arr[i] - minVal]] = arr[i];
  arr = output;
}

void radixSort(std::vector<int> &arr) {
  if (arr.empty())
    return;
  int maxVal = *std::max_element(arr.begin(), arr.end());
  int n = arr.size();
  int count[10];
  std::vector<int> buffer(n), *src = &arr, *dst = &buffer;
  for (int exp = 1; maxVal / exp > 0; exp *= 10) {
    std::fill(std::begin(count), std::end(count), 0);
    for (int i = 0; i < n; ++i) {
      int digit = ((*src)[i] / exp) % 10;
      count[digit]++;
    }
    for (int i = 1; i < 10; ++i) {
      count[i] += count[i - 1];
    }
    for (int i = n - 1; i >= 0; --i) {
      int digit = ((*src)[i] / exp) % 10;
      (*dst)[--count[digit]] = (*src)[i];
    }
    std::swap(src, dst);
  }
  if (src != &arr) {
    arr = buffer;
  }
}

void bucketSort(std::vector<int> &arr) {
  if (arr.empty())
    return;
  int maxVal = *std::max_element(arr.begin(), arr.end());
  int minVal = *std::min_element(arr.begin(), arr.end());
  int bucketCount = std::min(static_cast<int>(arr.size()), 1000);
  double range = static_cast<double>(maxVal - minVal + 1) / bucketCount;
  std::vector<std::vector<int>> buckets(bucketCount);
  for (int num : arr) {
    int bIdx = static_cast<int>((num - minVal) / range);
    if (bIdx >= bucketCount)
      bIdx = bucketCount - 1;
    buckets[bIdx].push_back(num);
  }
  for (int i = 0; i < bucketCount; ++i) {
    std::sort(buckets[i].begin(), buckets[i].end());
  }
  int index = 0;
  for (int i = 0; i < bucketCount; ++i) {
    for (int num : buckets[i]) {
      arr[index++] = num;
    }
  }
```

```
}

int main() {
  std::vector<size_t> testSizes = {1000, 10000, 100000, 1000000};
  for (size_t N : testSizes) {
    std::cout << "\n--- Input Size N = " << N << " ---\n";
    std::vector<int> original = generateTestCase(N, InputType::RANDOM);
    auto dataCS = original;
    auto start = std::chrono::steady_clock::now();
    countingSort(dataCS);
    auto end = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> timeCS = end - start;
    std::cout << "Counting Sort → Time: " << timeCS.count()
              << " ms | Validated: "
              << (std::is_sorted(dataCS.begin(), dataCS.end()) ? "Yes" : "No")
              << "\n";
    auto dataRS = original;
    start = std::chrono::steady_clock::now();
    radixSort(dataRS);
    end = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> timeRS = end - start;
    std::cout << "Radix Sort    → Time: " << timeRS.count()
              << " ms | Validated: "
              << (std::is_sorted(dataRS.begin(), dataRS.end()) ? "Yes" : "No")
              << "\n";
    auto dataBS = original;
    start = std::chrono::steady_clock::now();
    bucketSort(dataBS);
    end = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> timeBS = end - start;
    std::cout << "Bucket Sort   → Time: " << timeBS.count()
              << " ms | Validated: "
              << (std::is_sorted(dataBS.begin(), dataBS.end()) ? "Yes" : "No")
              << "\n";
  }
  return 0;
}
```

2.4 Terminal Output

```
$ ./linear_sort_bench

--- Input Size N = 1000 ---
Counting Sort → Time: 3.26934 ms | Validated: Yes
Radix Sort    → Time: 0.077753 ms | Validated: Yes
Bucket Sort   → Time: 0.14809 ms | Validated: Yes

--- Input Size N = 10000 ---
Counting Sort → Time: 3.3787 ms | Validated: Yes
Radix Sort    → Time: 0.780167 ms | Validated: Yes
Bucket Sort   → Time: 0.74668 ms | Validated: Yes

--- Input Size N = 100000 ---
Counting Sort → Time: 6.41015 ms | Validated: Yes
Radix Sort    → Time: 7.41758 ms | Validated: Yes
Bucket Sort   → Time: 6.25208 ms | Validated: Yes

--- Input Size N = 1000000 ---
Counting Sort → Time: 44.7324 ms | Validated: Yes
Radix Sort    → Time: 90.2502 ms | Validated: Yes
Bucket Sort   → Time: 82.8123 ms | Validated: Yes
```

3. Utilities

3.1 test.hpp

```hpp
#pragma once

#include <chrono>
#include <vector>

enum class InputType { RANDOM, SORTED, REVERSE_SORTED, DUPLICATES };
std::vector<int> generateTestCase(size_t size, InputType type);
```

3.2 test.cpp

```cpp
#include "test.hpp"
#include <algorithm>
#include <numeric>
#include <random>

std::vector<int> generateTestCase(size_t size, InputType type) {
  std::vector<int> arr(size);

  switch (type) {
  case InputType::RANDOM: {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> dist(1, 1000000);
    for (size_t i = 0; i < size; ++i)
      arr[i] = dist(gen);
    break;
  }
  case InputType::SORTED: {
    std::iota(arr.begin(), arr.end(), 1);
    break;
  }
  case InputType::REVERSE_SORTED: {
    std::iota(arr.rbegin(), arr.rend(), 1);
    break;
  }
  case InputType::DUPLICATES: {
    std::fill(arr.begin(), arr.end(), 42);
    break;
  }
  }
  return arr;
}
```